Concepts of Programming Languages

Examiner: Mira Mezini, Ragnar Mogk

TECHNISCHE UNIVERSITÄT DARMSTADT

2026-02-20

Full Name	
Registration Number (Matrikelnummer)	
Course of Study (Studiengang, e.g., Master CS)	
Department (Fachbereich, e.g., 20 Informatik)	
Signature (Unterschrift)	

Permitted Aids

- Pens with **document-proof** ink (blue or black). No pencils or eraseable pens and no correction fluid (Tipp-ex).
- Dictionary.
- **No** electronic aids are permitted (e.g., calculator, smartphone)! Turn off your phone and put it away.

Announcements

- Make sure that your copy of the exam is complete (15 pages).
- Fill out all fields above.
- Put your name and your enrollment ID (*Matrikelnummer*) on all pages of this exam (name stamps are OK).
- The time for solving this exam is 90 minutes.
- All questions of the exam must be answered in English.
- You are not allowed to remove the stapling.
- All tasks can be processed independently and in any order.
- You can use the backsides of the pages if you need more space. It is not allowed to use your own paper.
- If someone feels sick, notify us before the exam starts.
- If you feel disturbed (e.g., loud noises), notify us as soon as possible.
- **Sign your exam** to confirm your details and to acknowledge these announcements.

Task	1	2	3	4	5	6	7	8	9	Σ	Grade
Max.	2	5	16	4	7	12	5	6	6	63	—
Points											

LastName, FirstName: _____ EnrollmentID: ␣␣␣␣␣␣␣

Task 1: Auxiliary Techniques — 2 points

1a) Substitution in the Lambda Calculus — 2 points

The lambda calculus is a language with identifiers, function abstraction, and function application (`Id`, `Fun`, `App`). Terms in the lambda calculus are reduced by beta reduction

$$(\lambda x.e_0)\ e_1\ =_\beta\ e_0[e_1/x]$$

where $e[v/i]$ is the substitution of identifier i in an expression e with another expression v.

Apply beta reduction to reduce the following expressions until they can not be reduced further. Write each reduction step in a separate line:

- $(\lambda x.(\lambda y.x))\ z$

- $(\lambda f.f\ (\lambda x.x))\ (\lambda z.z\ y)$

LastName, FirstName: _____ EnrollmentID: ☐☐☐☐☐☐☐

Task 2: Functions — 5 points

2a) Function classification — 2 points

Classify the following functions as *first-order* or *higher-order* (1 point each).

1. Category of `sumInterval` is ...

```
def sumInterval2(n: Double,m : Double, f: Double => Double): Double =
    if (n == m ) f(m)
    else if (m > n)  f(n + sumInterval(n+ 1, m))
    else  f (m + sumInterval(n, m + 1))
```

2. Category of `(n => n % 2 == 0)` is ...

```
val k = (1 to 5000).filter(n => n % 2 == 0)
```

2b) Scoping — 3 points

1. Mention two reasons why dynamic scoping can causes problems and may lead to fragile software (2 points).

LastName, FirstName: _____ EnrollmentID: ☐☐☐☐☐☐☐

Task 3: Environments and evaluation order — 16 points

3a) Environment vs Substituition — 2 points

We introduced environments as an alternative for substitution. Why did we do this?

3b) Fixing an environment-based interpreter — 9 points

Consider the following fragment of an interpreter for the language `F1WAE` that supports `Let` expressions, arithmetics and first-order functions. The interpreter should implement static/lexical scoping. The shown interpreter contains 3 bugs.

```
type FunDefs = Map[String, FunDef]
type Env = Map[String, Int]

def interp(expr: F1WAE, funDefs: FunDefs, env: Env): Int = expr match {
  case Num(n) =>
    n

  case Sub(lhs, rhs) =>
    Sub(lhs,rhs)

  case Let(boundId, namedExpr, boundBody) =>
    interp(boundBody, funDefs, env)

  case Id(name) => env(name)

  case App(funName, argExpr) => funDefs(funName) match
    case FunDef(argName, body) =>
      interp(body, funDefs, env)
}
```

1. Identify and describe the 3 bugs: Where in the program does the bug occur, why is the given definition wrong?

2. Implement a fixed version of the interpreter above. It is sufficient if you reimplement those cases of the pattern match that need fixing. (Implement on the following page)

```
type FunDefs = Map[Symbol, FunDef]
type Emv = Map[Symbol, Int]

def interp(expr: F1WAE, funDefs: FunDefs, env: Emv): Int = expr match {

}
```

LastName, FirstName: _____ EnrollmentID: ☐☐☐☐☐☐☐

3c) Evaluation order — 5 points

Consider the following fragment of an interpreter for the WAE language :

```
def interp(expr: WAE, env: Env): Value = expr match {
  case Num(n) => n
  case Add(lhs, rhs) => interp(lhs) + interp(rhs)
  case Let(boundId, namedExpr, boundExpr) =>
    interp(boundExpr, env + (boundId - > EClosure(namedExpr, env)))
  case Id(name) => env(name)
}
```

1. Does this interpreter implement lazy evaluation (call-by-name) or eager evaluation (call-by-value)? Briefly justify your answer.

2. Change the interpreter such that it implements the other form of evaluation. It is sufficient if you reimplement those cases of the pattern match that need adaption.

```
def interp(expr: WAE): Int = expr match {

}
```

LastName, FirstName: _____ EnrollmentID: ☐☐☐☐☐☐☐

Task 4: Memory management — 4 points

4a) Garbage collected store — 4 points

Consider a program in an interpreter with state, mutable boxes, conditionals, first-class functions, let and arithmetic expressions. During the interpretation of the program, the garbage collector is executed. The state of the store and environment before garbage collection are shown below on the left ("Before GC").

Provide the state of the environment and store after executing a run of a (a) non-moving and (b) moving garbage collector (2 points each). Freed locations can be left blank.

Before GC:

Env = x → 15

Store:

Location	Value
0	Num(31)
1	Box(0)
2	Box(0)
3	Box(2)
4	Box(2)
5	Box(4)
6	Box(4)
7	Box(9)
8	Box(7)
9	Box(7)
10	Box(9)
11	Box(9)
12	Num(15)
13	Box(12)
14	Box(12)
15	Box(6)
16	Box(6)
17	Box(14)
18	

(a) non-moving GC:

Env =

Store:

Location	Value
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	
15	
16	
17	
18	

(b) moving GC:

Env =

Store:

Location	Value
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	
15	
16	
17	
18	

LastName, FirstName: _____ EnrollmentID: ☐☐☐☐☐☐☐

Task 5: Continuation-passing style — 7 points

5a) Transformation to CPS — 7 points

Consider the following definition of the function `inOrder` and its accompanying `Tree` data type in Scala:

```
abstract class Tree
case class Leaf(n: Int) extends Tree
case class Node(n: Int, left: Tree, right: Tree) extends Tree

def append(lhs: List[Int], rhs: List[Int]): List[Int] = lhs match {
  case Nil => rhs
  case x::xs => x::append(xs, rhs)
}

def inOrder(tree: Tree): List[Int] = tree match {
  case Leaf(n) => List(n)
  case Node(n, left, right) =>
    //inOrder(left) ++ List(n) ++ inOrder(right)
    val lhs = inOrder(left)
    val rhs = inOrder(right)
    val tempappend = append(List(n), rhs)
    append(lhs, tempappend)
}
```

Assuming the existence of a function `appendK`, which uses continuation-passing-style to append two lists, transform the `inOrder` function into an equivalent function `inOrderK` in continuation-passing style. In other words: Every call of `inOrderK` and `appendK` within `inOrderK` must be tail calls.

Hint: The structure of the `Node` case in `inOrder` suggests the order for the computations.

```
def appendK(lhs: List[Int], rhs: List[Int], k: List[Int] => Nothing): Nothing =
  lhs match {
    case Nil => k(rhs)
    case x::xs => appendK(xs, rhs, res => k(x::res))
  }

def inOrderK(tree: Tree, k: List[Int] => Nothing): Nothing =
```

LastName, FirstName: _____ EnrollmentID: ☐☐☐☐☐☐☐

Task 6: Lambda Calculus — 12 points

Note. This task is originally from 2014, when the course was still in Scheme, not Scala. The syntax is thus not really representative of current exams, but the topics are still relevant.

6a) Church Encoding of Natural Numbers — 3 points

In the lambda calculus, natural numbers (0, 1, 2, ...) are traditionally (Church-)encoded as follows:

```
;; Church encoding
(define zero (lambda (f) (lambda (x) x)))
(define succ (lambda (n) (lambda (f) (lambda (x) (f ((n f) x))))))
```

The following function then implements addition of two Church-encoded numbers by n times adding one to m:

```
(define add (lambda (m)
                    (lambda (n)
                            ((n succ) m))))
```

Suppose, someone proposed the following function as an alternative implementation of addition:

```
(define alt (lambda (m)
                    (lambda (n)
                            (lambda (f)
                                    (lambda (x)
                                            ((m f) ((n f) x))))))
```

Does the function `alt` really implement addition? Briefly justify your answer.

•

LastName, FirstName: _____ EnrollmentID: ☐☐☐☐☐☐☐

6b) Fix Point Combinators — 6 points

The defining property of a fix-point combinator `fix` is `(fix f)` * `(f (fix f))`, i.e. evaluation reaches a fixed point after a finite number of steps.

The function `fix` below is indeed a fixed-point combinator in Scheme (using λ instead of `lambda` for the sake of brevity):

```
(define (fix f)
        ((λ (x) (f (λ (v) ((x x) v))))
         (λ (x) (f (λ (v) ((x x) v))))))
```

This is proven by the following derivation:

	`(fix f)`
	`((λ (x) (f (λ (v) ((x x) v))))` `(λ (x) (f (λ (v) ((x x) v)))))`
	`(f (λ (v) (((λ (x) (f (λ (v) ((x x) v))))` `(λ (x) (f (λ (v) ((x x) v))))) v)))`
$\equiv$	`(f ((λ (x) (f (λ (v) ((x x) v))))` `(λ (x) (f (λ (v) ((x x) v))))))`
$\equiv$	`(f (fix f))`

Is the (simpler) function `sfix` below also a fixed-point combinator in Scheme? Prove or disprove this.

```
(define (sfix f)
        ((λ (x) (f (x x)))
         (λ (x) (f (x x))))
```

6c) Implementing Recursion through Fixed-Point Combinators — 3 points

Consider the following recursive function, which computes the length of a list:

```
(define (length l)
  (if (empty? l)
      0
      (+ 1 (length (rest l)))))
```

Use a pre-defined fixed-point combinator `fix` to transform `length` such that it no longer uses recursion:

```
(define
```

)

LastName, FirstName: _____ EnrollmentID: ☐☐☐☐☐☐☐

Task 7: Lazyness — 5 points

7a) Haskell infinite lists — 5 points

Consider the following definition of lists in Haskell.

1. Enumerate the first *6* elements of `secret`, i.e., state the output of `take 6 secret` (1 point).

```
secret :: [Int]
secret = 5 : 3 : map (2 +) secret
```

2. Enumerate the first *4* elements of `mystery`, i.e., state the output of `take 4 mystery` (1 points).

   Note: `++` concatenates two strings.

```
mystery :: [String]
mystery = "a" : zipWith (++) mystery mystery
```

3. Enumerate the first *6* elements of `enigma`, i.e., state the output of `take 6 enigma` (1 points).

```
enigma :: [Int]
enigma = 2 : 1 : zipWith (+) enigma (tail enigma)
```

4. Enumerate the first *5* elements of `unavowed`, i.e., state the output of `take 5 unavowed` (2 points).

   Note: `++` concatenates two strings.

```
unavowed :: [String]
unavowed = "a" : "d" : zipWith (++) (map (++ "c") unavowed) (tail unavowed)
```

LastName, FirstName: _____ EnrollmentID: ☐☐☐☐☐☐☐

Task 8: Lazyness — 6 points

8a) Eager vs. Lazy Evaluation — 2 points

Consider an interpreter with first-class functions, Let, LetRec, arithmetic expressions, and static scoping. There are two variants of this interpreter one implementing a language with eager evaluation and another a language with lazy evaluation (**without caching**). For lazy evaluation, assume that there is a strict point immediately after the program. Always use static scoping.

For the following program, **how many additions** does the interpreter compute? (That is, how often does the interpreter need to evaluate an `Add` expression to a value?) (1 point each)

```
Let("f", Fun("x", Add("x", "x")),
  Let("y", Add(1, Add(2, 3)),
    App("f", "y")))
```

Eager Evaluation:

Lazy Evaluation (without caching):

8b) Eager vs. Lazy Evaluation — 4 points

Assume the same setting as in the previous task. Decide for each of the following two programs whether they will raise an error in a language with eager evaluation and in a language with lazy evaluation (**without caching**), respectively.

If it terminates without an error, what is the program's result? If it does not terminate, what kind of error does the interpreter raise? (1 point each)

Consider all given expressions as syntactically correct. If necessary, "b" is wrapped in Id("b") and 42 in Num(42) via an implicit conversion.

```
Let("g", Fun("x", 1),
  LetRec("f", Fun("x", App("f", "x")),
    App("g", App("f", 20))
  )
)
```

Eager Evaluation:

Lazy Evaluation (without caching):

```
LetRec("x", 7,
  Let("y",
    Fun("n", Add("n", "x")),
    App("y", 5))
)
```

Eager Evaluation:

Lazy Evaluation (without caching):

LastName, FirstName: _____ EnrollmentID: ☐☐☐☐☐☐☐

Task 9: De-Bruijn Indices — 6 points

De-Bruijn indices are a way to avoid naming variables. Instead of referring to an identifier by name, we count "how many scopes ago" the identifier was defined.

For example `Let(Num(3), Let(Num(4), Add(Id(0), Id(1))) )` evaluates to `Add(Num(4), Num(3))` (and finally to `Num(7)`).

Consider the following program written in the de-Bruijn variant of our language:

```
Let( Num(1),
  Let( Add( Id(0), Num(2) ),
    Add(
      Let( Num(3),
        Add( Id(0), Id(1) )),
      Id(0)
    )
  )
)
```

What does this program evaluate to? (2 points)

•

Transform the above program into an equivalent program using named bindings. The resulting program must no longer use de-Bruijn indices (`Id(0)`) but should use named variables (`Id("name")`) instead:

```
Let("outer",

)
```